

PROMETEO



PowerChat

Técnico Superior Desarrollo de aplicaciones Web / Online

Javier García Manotas

Tutor del TFG: Raúl Albiol

ABSTRACT

PowerChat es una aplicación web diseñada para la gestión profesional de conversaciones de WhatsApp en entornos empresariales multiagente. El sistema permite que varios agentes trabajen simultáneamente sobre conversaciones entrantes, con actualización en tiempo real, control de estados y sistema de asignaciones. Se ha desarrollado utilizando Node.js con Express en el backend, Vue 3 con Composition API en el frontend, MySQL como base de datos relacional gestionada a través de Prisma ORM, y Socket.IO para la comunicación en tiempo real mediante WebSockets. La aplicación implementa un sistema completo de autenticación basado en JWT, control de acceso por roles (Administrador, Supervisor y Agente), integración con proveedores de mensajería externos mediante webhooks, y un panel de estadísticas con visualizaciones gráficas. El resultado es una herramienta funcional, segura y responsiva que aborda los elementos fundamentales de las plataformas profesionales de atención al cliente a través de WhatsApp.

PowerChat is a web application designed for the professional management of WhatsApp conversations in multi-agent business environments. The system enables multiple agents to work simultaneously on incoming conversations, with real-time updates, status control, and an assignment system. It was developed using Node.js with Express on the backend, Vue 3 with Composition API on the frontend, MySQL as the relational database managed through Prisma ORM, and Socket.IO for real-time communication via WebSockets. The application implements a complete JWT-based authentication system, role-based access control (Administrator, Supervisor, and Agent), integration with external messaging providers through webhooks, and a statistics dashboard with graphical visualizations. The result is a functional, secure, and responsive tool that addresses the fundamental elements of professional customer service platforms through WhatsApp.

ÍNDICES

ABSTRACT	2
ÍNDICES.....	3
INTRODUCCIÓN	5
JUSTIFICACIÓN DEL PROYECTO	6
Estado del arte y comparativa	6
Público objetivo.....	7
OBJETIVOS	8
R01 – El sistema debe permitir la autenticación y gestión de usuarios	8
R02 – El sistema debe gestionar conversaciones de WhatsApp.....	9
R03 – El sistema debe permitir el envío y recepción de mensajes en tiempo real	10
R04 – El sistema debe recibir eventos externos mediante webhooks	10
R05 – El sistema debe ofrecer un panel de estadísticas	10
R06 – La interfaz debe ser responsiva y adaptarse a múltiples dispositivos	11
DESCRIPCIÓN	12
Arquitectura de la solución	12
Casos de uso.....	13
DISEÑOS	15
Diagrama Entidad-Relación.....	15
Diagrama de la base de datos	16
Interfaces de usuario	17
Pantalla de inicio de sesión.....	17
Bandeja de conversaciones	17
Cabecera de conversación y asignación.....	17
Burbujas de mensaje	17
Panel de estadísticas (Dashboard)	18
Gestión de usuarios	18
Perfil de usuario	18
TECNOLOGÍA.....	19
Backend.....	19
Frontend	20
Herramientas de desarrollo	20
Despliegue.....	21
METODOLOGÍA	22
Metodología utilizada: Desarrollo iterativo incremental.....	22
Planificación y presupuesto de horas	23

README y GIT.....	24
TRABAJOS FUTUROS	25
CONCLUSIONES.....	26
REFERENCIAS.....	27

INTRODUCCIÓN

PowerChat es una aplicación web diseñada para la gestión profesional de conversaciones de WhatsApp en empresas. En un contexto donde WhatsApp se ha consolidado como uno de los canales de comunicación más utilizados entre empresas y clientes, surge la necesidad de herramientas que permitan gestionar estas conversaciones de forma estructurada, especialmente cuando intervienen varios empleados o agentes de atención al cliente.

El sistema permite que varios agentes trabajen simultáneamente sobre conversaciones entrantes, ofreciendo actualización en tiempo real, control de estados de las conversaciones (abierta, pendiente, cerrada) y un sistema completo de asignaciones entre agentes. Todo ello desde una interfaz web responsiva que se adapta a dispositivos de escritorio, tableta y móvil.

Los principales requisitos que debe cumplir el proyecto son los siguientes: permitir la gestión multiagente de conversaciones de WhatsApp, ofrecer comunicación en tiempo real mediante WebSockets, implementar un sistema de autenticación y autorización basado en roles (Administrador, Supervisor y Agente), integrar un proveedor externo de mensajería a través de webhooks, proporcionar un panel de estadísticas para la supervisión del rendimiento, y garantizar una experiencia de usuario fluida y responsiva en todos los dispositivos.

El objetivo es desarrollar una herramienta similar a las plataformas profesionales de atención al cliente como Whanmo, ManyContacts o Respond.io, pero centrada exclusivamente en WhatsApp y con un enfoque técnico académico que demuestre las competencias adquiridas durante el Grado Superior en Desarrollo de Aplicaciones Web.

JUSTIFICACIÓN DEL PROYECTO

La idea del proyecto surge de la experiencia profesional del autor en una empresa que desarrolla una plataforma similar (whanmo.com). Esta experiencia ha permitido conocer de primera mano la creciente importancia de WhatsApp como canal de comunicación empresarial y las carencias que aún existen en muchas organizaciones a la hora de gestionarlo de forma profesional.

Actualmente existen diversas herramientas en el mercado que permiten la gestión multiagente de conversaciones de WhatsApp. Entre las más destacadas se encuentran:

Estado del arte y comparativa

Whanmo (whanmo.com): plataforma SaaS española especializada en la gestión de WhatsApp para equipos comerciales. Ofrece bandeja compartida, asignación automática, etiquetas y seguimiento de conversaciones. Orientada a PYMEs con planes de pago mensual.

ManyContacts: herramienta que permite gestionar WhatsApp desde un CRM compartido. Integra funciones de embudo de ventas, notas internas y sistema de tareas. Orientada a equipos de ventas con necesidades de seguimiento comercial.

Respond.io: plataforma omnicanal que soporta múltiples canales de mensajería (WhatsApp, Telegram, Facebook Messenger, etc.). Ofrece automatizaciones avanzadas, chatbots y análisis detallados. Orientada a empresas medianas y grandes con necesidades multicanal.

Sin embargo, muchas empresas todavía gestionan WhatsApp de forma poco estructurada cuando intervienen varios empleados, utilizando dispositivos compartidos o cuentas sin control interno. Esto genera problemas de organización, reparto de trabajo y trazabilidad.

PowerChat propone una implementación técnica que aborda los elementos fundamentales de estas plataformas profesionales: gestión estructurada de conversaciones, asignación entre agentes, control de estados y sincronización en tiempo real. A diferencia de las soluciones comerciales, PowerChat se desarrolla

como un proyecto académico de código abierto que puede servir como base para implementaciones personalizadas.

Público objetivo

El sistema está dirigido a empresas que utilizan WhatsApp como canal de comunicación y necesitan organizar la gestión entre varios agentes. Aporta especial valor en organizaciones de tres o más empleados, donde la coordinación y el reparto de conversaciones requieren una herramienta estructurada. Es aplicable a distintos sectores, dado el uso generalizado de WhatsApp en el entorno empresarial español e internacional.

OBJETIVOS

A continuación, se presenta el desglose RFTP (Requisitos, Funciones, Tareas y Pruebas) del proyecto PowerChat. Este desglose permite entender de forma jerárquica qué debe hacer el sistema, cómo se descompone cada requisito en funcionalidades concretas, qué tareas de implementación implica cada una y cómo se verifica su correcto funcionamiento.

R01 – El sistema debe permitir la autenticación y gestión de usuarios

R01F01 – El usuario debe poder iniciar sesión con email y contraseña.

R01F01T01 – Implementar endpoint POST /auth/login con validación de credenciales y generación de token JWT (expiración 24h).

R01F01T01P01 – Verificar login correcto con credenciales válidas y recepción de token.

R01F01T01P02 – Verificar rechazo con credenciales inválidas y mensaje de error apropiado.

R01F01T02 – Implementar rate limiting de 3 intentos cada 15 segundos para prevenir ataques de fuerza bruta.

R01F01T02P01 – Verificar que al cuarto intento consecutivo se devuelve error 429.

R01F02 – El administrador debe poder gestionar usuarios (crear, editar, desactivar).

R01F02T01 – Implementar endpoints CRUD en /users con control de acceso ADMIN.

R01F02T01P01 – Crear un usuario nuevo y verificar que aparece en el listado.

R01F02T01P02 – Verificar que un usuario con rol AGENT no puede acceder a /users.

R01F03 – El sistema debe soportar tres roles: ADMIN, SUPERVISOR y AGENT.

R01F03T01 – Implementar middleware requireRole para verificar permisos por rol en cada endpoint.

R01F03T01P01 – Verificar que cada rol accede solo a las funcionalidades permitidas.

R02 – El sistema debe gestionar conversaciones de WhatsApp

R02F01 – Visualizar un listado de conversaciones con filtros por estado y alcance.

R02F01T01 – Implementar GET /conversations con paginación, filtros de scope (mine/unassigned/all) y status (OPEN/PENDING/CLOSED).

R02F01T01P01 – Verificar que un agente solo ve sus conversaciones asignadas y las no asignadas.

R02F02 – Permitir la asignación y desasignación de conversaciones entre agentes.

R02F02T01 – Implementar endpoints assign-to-me, assign y unassign con lógica RBAC.

R02F02T01P01 – Verificar que un agente puede autoasignarse una conversación no asignada.

R02F02T01P02 – Verificar que un supervisor puede asignar cualquier conversación a cualquier agente activo.

R02F03 – Permitir cambiar el estado de las conversaciones.

R02F03T01 – Implementar PATCH /conversations/:id/status.

R02F03T01P01 – Cambiar estado de OPEN a PENDING y verificar la actualización en tiempo real.

R02F04 – Permitir editar el nombre de contacto de las conversaciones.

R02F04T01 – Implementar PATCH /conversations/:id/contact.

R03 – El sistema debe permitir el envío y recepción de mensajes en tiempo real

R03F01 – Visualizar el historial de mensajes de cada conversación con paginación por cursor.

R03F01T01 – Implementar GET /conversations/:id/messages con cursor-based pagination.

R03F01T01P01 – Cargar mensajes iniciales y verificar la carga de mensajes anteriores al hacer scroll.

R03F02 – Enviar mensajes a través del proveedor externo de mensajería.

R03F02T01 – Implementar POST /conversations/:id/messages con integración al API del proveedor.

R03F02T01P01 – Enviar un mensaje y verificar que se refleja en la conversación.

R03F03 – Recibir mensajes y actualizaciones de estado en tiempo real via WebSockets.

R03F03T01 – Implementar Socket.IO con autenticación JWT y sistema de salas por conversación y rol.

R03F03T01P01 – Verificar que un mensaje entrante aparece instantáneamente sin recargar la página.

R04 – El sistema debe recibir eventos externos mediante webhooks

R04F01 – Procesar webhooks del proveedor de mensajería de forma segura.

R04F01T01 – Implementar POST /webhooks/provider con verificación de cabecera secreta (X-Webhook-Secret) y procesamiento asíncrono.

R04F01T01P01 – Enviar un webhook simulado y verificar la creación del mensaje en la base de datos.

R05 – El sistema debe ofrecer un panel de estadísticas

R05F01 – Mostrar KPIs y gráficos de rendimiento para supervisores y administradores.

R05F01T01 – Implementar GET /stats con filtros de fecha y métricas por agente.

R05F01T01P01 – Verificar que los KPIs reflejan los datos reales del sistema.

R05F02 – Permitir drill-down a las conversaciones de cada agente.

R05F02T01 – Implementar GET /stats/users/:userId/conversations.

R06 – La interfaz debe ser responsiva y adaptarse a múltiples dispositivos

R06F01 – Diseñar layout adaptativo para escritorio, tableta y móvil.

R06F01T01 – Implementar breakpoints responsive (desktop ≥ 1024 px, tablet 768-1023px, móvil < 768 px).

R06F01T01P01 – Verificar la navegación en cada tamaño de pantalla.

DESCRIPCIÓN

En esta sección se describen los diagramas y la arquitectura de la solución propuesta para PowerChat.

Arquitectura de la solución

PowerChat sigue una arquitectura cliente-servidor basada en tres capas principales:

La capa de presentación (frontend) es una Single Page Application (SPA) desarrollada con Vue 3 y desplegada como contenido estático. Se comunica con el backend a través de peticiones HTTP REST y conexiones WebSocket persistentes mediante Socket.IO.

La capa de lógica de negocio (backend) es un servidor Node.js con Express que expone una API REST y gestiona las conexiones WebSocket. Se encarga de la autenticación (JWT), la autorización por roles (RBAC), el procesamiento de webhooks entrantes del proveedor de mensajería, y la difusión de eventos en tiempo real a los clientes conectados.

La capa de datos es una base de datos MySQL gestionada a través de Prisma ORM, que almacena usuarios, conversaciones, mensajes y estados de lectura.

Además, el sistema se integra con un proveedor externo de mensajería que actúa como puente con la API de WhatsApp Business. El proveedor envía webhooks al backend cuando se reciben mensajes o cambian estados, y el backend utiliza la API del proveedor para enviar mensajes salientes.

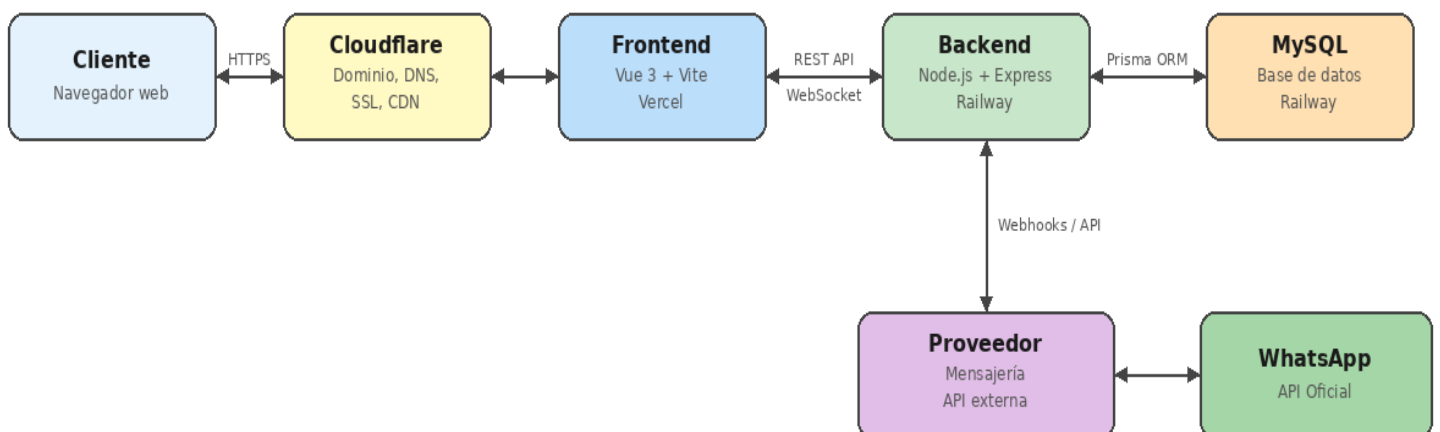


Figura 1. Arquitectura de la solución de PowerChat

Casos de uso

El sistema contempla tres actores principales (Agente, Supervisor/Administrador y Sistema/Proveedor externo) con los siguientes casos de uso principales:

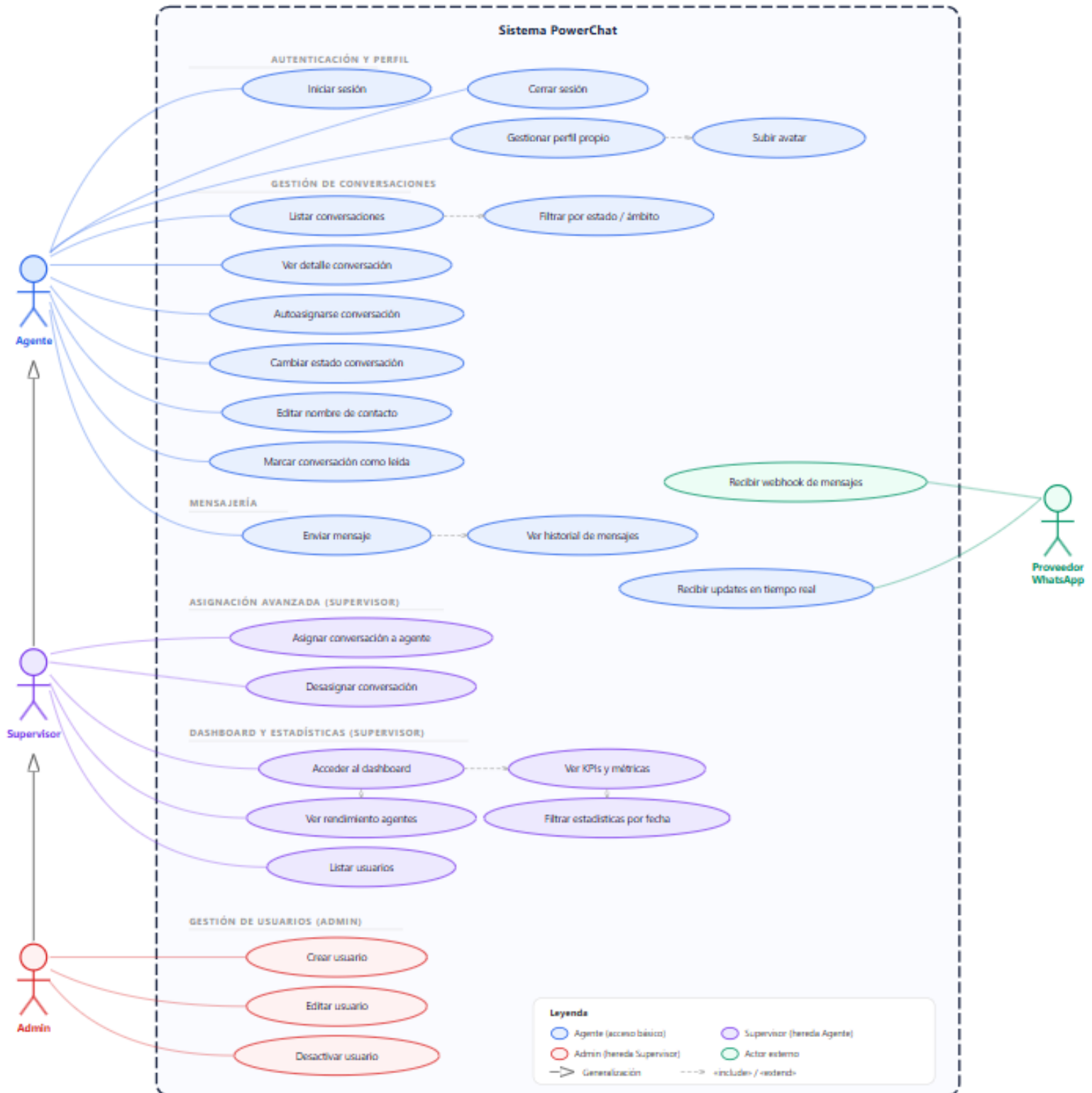


Figura 2. Diagrama de casos de uso UML

A continuación, se detallan los casos de uso más representativos:

Caso de uso: Gestionar conversaciones	
DESCRIPCIÓN: El agente visualiza, filtra y gestiona las conversaciones de WhatsApp asignadas o no asignadas, pudiendo auto asignarse conversaciones, cambiar su estado y enviar mensajes.	
PRECONDICIONES: Usuario autenticado con rol mínimo AGENT	POSTCONDICIONES: - Conversación asignada al agente - Estado actualizado - Mensaje enviado
DATOS ENTRADA: - Id conversación - Texto del mensaje - Nuevo estado	DATOS SALIDA: - Conversación actualizada - Mensaje creado con estado - Evento WebSocket emitido
TABLAS: - Conversation - Message - ConversationUserState	INTERFACES: - ConversationList.vue - MessageThread.vue - MessageInput.vue - ConversationHeader.vue

Tabla 1. Caso de uso: Gestionar conversaciones

Caso de uso: Consultar estadísticas	
DESCRIPCIÓN: El supervisor o administrador consulta los KPIs del sistema, gráficos de actividad y rendimiento por agente, con filtros de rango de fechas.	
PRECONDICIONES: Usuario autenticado con rol SUPERVISOR o ADMIN	POSTCONDICIONES: - Datos estadísticos mostrados - Gráficos renderizados
DATOS ENTRADA: - Fecha inicio - Fecha fin - Id agente	DATOS SALIDA: - KPIs (abiertas, pendientes, resueltas) - Gráficos por día Métricas por agente
TABLAS: - Conversation - Message - User	INTERFACES: - DashboardView.vue - KpiCard.vue - AgentTable.vue

Tabla 2. Caso de uso: Consultar estadísticas

DISEÑOS

Diagrama Entidad-Relación

La base de datos de PowerChat se compone de cuatro tablas principales interrelacionadas:

La tabla User almacena la información de los agentes, supervisores y administradores del sistema. Incluye campos para email (único), contraseña cifrada con scrypt, rol (ADMIN, SUPERVISOR o AGENT), datos personales, avatar y estado de activación (borrado lógico).

La tabla Conversation representa cada conversación de WhatsApp. Contiene un identificador externo del proveedor de mensajería, el teléfono del cliente, un nombre de contacto editable, el estado actual (OPEN, PENDING o CLOSED), una relación opcional con el usuario asignado, y campos de seguimiento del último mensaje.

La tabla Message almacena cada mensaje individual con su identificador externo, dirección (IN para entrantes, OUT para salientes), estado de entrega (PENDING, SENT, RECEIVED, READ, ERROR, DELETED), contenido de texto, marcas temporales y relaciones con la conversación y el usuario que envió el mensaje (para mensajes salientes).

La tabla ConversationUserState gestiona el estado de lectura por usuario y conversación, almacenando la fecha del último mensaje leído por cada usuario en cada conversación.

[INSERTAR AQUÍ: Diagrama E/R de la base de datos]

Diagrama de la base de datos

A continuación, se muestra el esquema detallado de la base de datos con todos los campos, tipos de datos, índices y relaciones:

Tabla	Campo	Tipo	Descripción
User	id	Int (PK)	Identificador autoincremental
User	email	String (unique)	Correo electrónico único
User	password	String	Hash scrypt (salt:hash)
User	role	Enum	ADMIN, SUPERVISOR, AGENT
User	firstName, lastName	String	Nombre y apellidos
User	active	Boolean	Borrado lógico (default: true)
User	avatarUrl	String?	URL de la imagen de perfil

Conversation	id	Int (PK)	Identificador autoincremental
Conversation	externalId	String (unique)	ID del proveedor de mensajería
Conversation	customerPhone	String?	Teléfono del cliente
Conversation	contactName	String?	Nombre de contacto editable
Conversation	status	Enum	OPEN, PENDING, CLOSED
Conversation	assignedToId	Int? (FK)	Agente asignado
Conversation	lastMessageAt	DateTime?	Fecha último mensaje

Message	id	Int (PK)	Identificador autoincremental
Message	externalId	String (unique)	ID del proveedor
Message	direction	Enum	IN (entrante), OUT (saliente)
Message	state	Enum	PENDING, SENT, RECEIVED, READ, ERROR, DELETED
Message	text	Text?	Contenido del mensaje
Message	conversationId	Int (FK)	Conversación asociada
Message	sentById	Int? (FK)	Agente que envió (mensajes OUT)

ConvUserState	id	Int (PK)	Identificador autoincremental
ConvUserState	conversationId	Int (FK)	Conversación
ConvUserState	userId	Int (FK)	Usuario
ConvUserState	lastReadAt	DateTime?	Última lectura

Tabla 3. Esquema detallado de la base de datos

Interfaces de usuario

A continuación se muestran las capturas de pantalla de las principales interfaces de la aplicación, mostrando su funcionamiento en diferentes dispositivos y tamaños de pantalla.

Pantalla de inicio de sesión

La pantalla de login presenta un formulario centrado con campos para email y contraseña, botón de acceso y decoración geométrica de fondo. Incluye control de visibilidad de contraseña y mensajes de error animados.

[INSERTAR AQUÍ: Captura de pantalla de LoginView (escritorio y móvil)]

Bandeja de conversaciones

La vista principal muestra un layout de dos columnas en escritorio: a la izquierda el listado de conversaciones con filtros (alcance y estado), y a la derecha el hilo de mensajes de la conversación seleccionada. En móvil se muestra como vista de panel único con navegación entre lista y chat.

[INSERTAR AQUÍ: Captura de MainView en escritorio (vista completa con conversación abierta)]

[INSERTAR AQUÍ: Captura de MainView en móvil (lista de conversaciones)]

[INSERTAR AQUÍ: Captura de MainView en móvil (hilo de mensajes)]

Cabecera de conversación y asignación

La cabecera muestra el nombre del contacto (editable inline), teléfono, badge de estado, selector de agente para asignación y selector de estado. En dispositivos pequeños, las acciones se agrupan en un menú hamburguesa.

[INSERTAR AQUÍ: Captura de ConversationHeader con dropdown de asignación abierto]

Burbujas de mensaje

Los mensajes se muestran con estilo similar a WhatsApp: burbujas verdes para mensajes salientes y blancas para entrantes. Cada burbuja incluye el texto, la hora y un icono de estado de entrega (reloj, tick simple, doble tick gris, doble tick azul o error).

[INSERTAR AQUÍ: Captura de MessageBubble mostrando diferentes estados de mensaje]

Panel de estadísticas (Dashboard)

El dashboard muestra tarjetas KPI con métricas clave (conversaciones abiertas, sin asignar, asignadas, pendientes, resueltas), un gráfico de dona por estado, gráficos de línea de actividad diaria, y una tabla de rendimiento por agente con funcionalidad de drill-down.

[INSERTAR AQUÍ: Captura de DashboardView completa]

[INSERTAR AQUÍ: Captura del modal de drill-down de conversaciones de un agente]

Gestión de usuarios

La vista de gestión de usuarios (solo administradores) muestra una tabla con avatar, nombre, email, rol y estado de cada usuario. Incluye modales para crear y editar usuarios.

[INSERTAR AQUÍ: Captura de UsersView con tabla de usuarios]

[INSERTAR AQUÍ: Captura del modal de creación/edición de usuario]

Perfil de usuario

La vista de perfil permite ver la información del usuario autenticado, subir un avatar (arrastrar y soltar o clic), y alternar entre modo claro y oscuro.

[INSERTAR AQUÍ: Captura de ProfileView (modo claro y modo oscuro)]

TECNOLOGÍA

En este apartado se recogen las tecnologías que he utilizado para desarrollar PowerChat, explicando para qué sirve cada una y por qué la elegí.

Backend

Node.js: es el entorno que permite ejecutar JavaScript fuera del navegador, directamente en el servidor. Lo elegí porque me permite usar el mismo lenguaje (JavaScript) tanto en el backend como en el frontend, lo cual simplifica mucho el desarrollo. Además, funciona muy bien para aplicaciones en tiempo real como PowerChat porque gestiona muchas conexiones simultáneas de forma eficiente.

Express v5: es el framework más popular de Node.js para crear APIs. Facilita mucho la definición de rutas (endpoints), la gestión de peticiones HTTP y la aplicación de middlewares como la autenticación o el rate limiting. En PowerChat lo uso para montar toda la API REST del proyecto (19 endpoints en total).

Prisma ORM: es una herramienta que facilita trabajar con bases de datos desde Node.js. En lugar de escribir consultas SQL a mano, defines el esquema de tus tablas en un fichero y Prisma genera automáticamente las funciones para hacer consultas, inserciones, actualizaciones, etc. También gestiona las migraciones de la base de datos. Lo uso para todas las operaciones con MySQL.

Socket.IO v4: es la librería que uso para la comunicación en tiempo real. Funciona sobre WebSockets y permite que el servidor envíe datos al navegador instantáneamente sin que el cliente tenga que estar preguntando continuamente. En PowerChat lo uso para que los mensajes nuevos, los cambios de estado y las asignaciones aparezcan al momento en la pantalla de todos los agentes conectados.

JSON Web Token (JWT): es el sistema que utilizo para la autenticación. Cuando un usuario hace login, el servidor le genera un token que incluye su id y su rol. Ese token se envía en cada petición posterior para identificar al usuario sin necesidad de mantener sesiones en el servidor. Ya tenía experiencia previa trabajando con JWT en mi entorno profesional, lo que me permitió implementarlo de forma rápida y segura.

MySQL: es la base de datos relacional que almacena toda la información del sistema: usuarios, conversaciones, mensajes y estados de lectura. La elegí por ser una de las bases de datos más utilizadas, con gran estabilidad y buen rendimiento. Además, ya la conocía del ciclo formativo.

Frontend

Vue 3 (Composition API): es el framework de JavaScript que uso para construir toda la interfaz de usuario. Con Vue puedes crear componentes reutilizables y la página se actualiza automáticamente cuando cambian los datos. Uso la Composition API con la sintaxis `<script setup>`, que es la forma más moderna y limpia de organizar el código en Vue.

Vite: es la herramienta que uso para arrancar el servidor de desarrollo y para generar el build de producción del frontend. Es mucho más rápido que Webpack y los cambios que haces en el código se reflejan al instante en el navegador sin recargar la página.

Pinia: es el gestor de estado oficial de Vue 3 (sustituye a Vuex). Lo uso para guardar datos que necesitan compartirse entre varios componentes, como la información del usuario logueado, la lista de conversaciones, las estadísticas del dashboard y el tema claro/oscuro.

Tailwind CSS: es un framework de CSS que funciona con clases de utilidad. En vez de escribir hojas de estilo separadas, aplicas las clases directamente en el HTML. Esto me ha permitido maquetar toda la interfaz de forma rápida, incluyendo el modo oscuro y la adaptación a diferentes tamaños de pantalla (móvil, tableta y escritorio).

Chart.js: es una librería para crear gráficos en la web. La uso en el dashboard para pintar el gráfico de dona con el reparto de conversaciones por estado y los gráficos de líneas con la actividad diaria.

Herramientas de desarrollo

Git: lo uso para el control de versiones del código. Me permite llevar un historial de todos los cambios y volver atrás si algo falla.

Postman: herramienta que he usado para probar todos los endpoints de la API durante el desarrollo. Tengo colecciones organizadas por funcionalidad (auth, conversaciones, usuarios, etc.).

Visual Studio Code: el editor de código que he usado durante todo el proyecto, con extensiones para Vue, Prisma y Tailwind CSS.

Despliegue

Para poner la aplicación en producción y que sea accesible desde internet, he utilizado los siguientes servicios:

Railway: es una plataforma de despliegue en la nube que he utilizado para alojar tanto el backend (servidor Node.js con Express) como la base de datos MySQL. Railway permite desplegar aplicaciones directamente desde un repositorio de Git y gestiona automáticamente las variables de entorno, los certificados SSL y el escalado del servicio. La elección de Railway se debe a su facilidad de uso, su plan gratuito para proyectos pequeños y su soporte nativo para bases de datos MySQL.

Vercel: es la plataforma donde he desplegado el frontend (la SPA de Vue). Vercel está especialmente optimizada para aplicaciones frontend y ofrece una CDN global que sirve los archivos estáticos desde el servidor más cercano al usuario. El despliegue se hace automáticamente cada vez que subo cambios al repositorio de Git.

Cloudflare: lo he utilizado para la gestión del dominio y la configuración de DNS. Cloudflare actúa como intermediario entre el usuario y los servidores, proporcionando una capa adicional de seguridad (protección DDoS) y mejorando la velocidad de carga gracias a su sistema de caché. Además, gestiona los certificados SSL del dominio personalizado.

La arquitectura de despliegue queda así: el usuario accede al dominio configurado en Cloudflare, que redirige las peticiones del frontend a Vercel y las peticiones de la API al backend en Railway, donde también se encuentra la base de datos MySQL.

METODOLOGÍA

Metodología utilizada: Desarrollo iterativo incremental

Para el desarrollo de PowerChat se ha seguido una metodología iterativa e incremental. Esta aproximación consiste en construir el sistema de forma progresiva, añadiendo funcionalidades completas una a una. En cada iteración se completa un ciclo de análisis, diseño, implementación y pruebas para una funcionalidad concreta, que se integra al sistema antes de abordar la siguiente.

Se ha elegido esta metodología por las siguientes razones: el proyecto ha sido desarrollado por un único programador, lo que hace innecesario el overhead de marcos más formales como Scrum; permite obtener un producto funcional desde las primeras iteraciones; facilita la detección temprana de problemas al probar cada funcionalidad de forma aislada; y ofrece flexibilidad para reajustar el alcance según el tiempo disponible.

Las iteraciones principales del proyecto han sido las siguientes, siempre completando primero el backend de cada funcionalidad y luego su frontend correspondiente:

Iteración 1: Configuración del entorno. Se estableció la estructura del proyecto (backend con Express y Prisma, frontend con Vue y Vite), se configuró la base de datos MySQL, se implementó el sistema de autenticación con JWT y se preparó la pantalla de login.

Iteración 2: Backend de conversaciones y eventos WebSocket. Se implementaron todos los endpoints relacionados con las conversaciones (listado con filtros, asignaciones, estados, mensajes), la integración con el proveedor de mensajería mediante webhooks, y el sistema de eventos en tiempo real con Socket.IO.

Iteración 3: Frontend del listado de conversaciones. Se desarrolló la vista principal con el layout de dos columnas, el listado de conversaciones con filtros, el hilo de mensajes con paginación por cursor, las burbujas estilo WhatsApp, la cabecera con asignación y cambio de estado, y la integración con los eventos WebSocket para la actualización en tiempo real.

Iteración 4: Backend del perfil de usuario. Se implementaron los endpoints para obtener el perfil del usuario autenticado y para la subida de avatar con validación de formato y tamaño.

Iteración 5: Frontend del perfil de usuario. Se desarrolló la vista de perfil con la información del usuario, la subida de avatar con drag-and-drop, y el selector de tema claro/oscuro.

Iteración 6: Backend del CRUD de usuarios. Se implementaron los endpoints de listado, creación, edición y desactivación de usuarios con control de acceso exclusivo para administradores.

Iteración 7: Frontend del CRUD de usuarios. Se desarrolló la tabla de usuarios con modales de creación y edición, validación de formularios y feedback visual.

Iteración 8: Backend del dashboard. Se implementaron los endpoints de estadísticas con filtros de rango de fechas, KPIs, métricas por agente y drill-down a conversaciones.

Iteración 9: Frontend del dashboard. Se desarrolló el panel de estadísticas con tarjetas KPI, gráficos de dona y líneas (Chart.js), tabla de rendimiento por agente con drill-down, y adaptación responsiva.

Planificación y presupuesto de horas

El proyecto se ha desarrollado con una dedicación total aproximada de 100 horas, distribuidas de la siguiente manera y en el mismo orden en el que se fueron implementando las funcionalidades:

Fase / Tarea	Horas estimadas	Horas reales	Desviación
Análisis y diseño	8	8	0h
Configuración del entorno y autenticación	3	3	0h
Backend: Conversaciones, mensajes, webhooks y WebSocket	20	22	+2h
Frontend: Bandeja de conversaciones y tiempo real	20	22	+2h
Backend: Perfil de usuario	4	4	0h
Frontend: Perfil de usuario	4	4	0h

Backend: CRUD de usuarios	6	6	0h
Frontend: CRUD de usuarios	6	6	0h
Backend: Dashboard y estadísticas	6	6	0h
Frontend: Dashboard y estadísticas	8	8	0h
Responsividad y adaptación móvil	5	4	-1h
Despliegue (Railway, Vercel, Cloudflare)	4	3	-1h
Pruebas y corrección de errores	4	4	0h
Documentación	2	2	0h
TOTAL	100	102	+2h

Tabla 4. Presupuesto de horas del proyecto

Las principales desviaciones se han producido en la parte de conversaciones, tanto en el backend como en el frontend, que resultó más compleja de lo previsto debido a la lógica de filtrado por roles, la paginación por cursor y la integración completa de los eventos WebSocket. Estas desviaciones se compensaron parcialmente con un menor tiempo del esperado en la adaptación responsive (Tailwind CSS facilitó notablemente esta tarea) y en el despliegue (ya tenía experiencia previa con Railway y Vercel).

README y GIT

El código fuente del proyecto se gestiona con Git, con un repositorio que incluye el código del backend y del frontend en la misma estructura. El fichero README.md contiene instrucciones de instalación, configuración de variables de entorno y comandos para ejecutar el proyecto en desarrollo y producción.

[INSERTAR AQUÍ: Captura de pantalla del historial de commits en Git]

TRABAJOS FUTUROS

A continuación se describen las líneas de ampliación y mejora proyectadas para futuras versiones de PowerChat:

Soporte multimedia: añadir la capacidad de enviar y recibir imágenes, vídeos, documentos y notas de voz, además de mensajes de texto. Esto requiere ampliar el modelo de datos de mensajes y la integración con el proveedor de mensajería para soportar diferentes tipos de contenido.

Respuestas rápidas y plantillas: implementar un sistema de respuestas predefinidas que los agentes puedan utilizar para agilizar la atención al cliente. Incluiría un editor de plantillas con variables dinámicas (nombre del cliente, número de referencia, etc.).

Notas internas: permitir que los agentes añadan notas privadas a las conversaciones, visibles solo para el equipo interno, para facilitar la coordinación y el traspaso de información entre agentes.

Sistema de etiquetas: añadir la posibilidad de clasificar conversaciones mediante etiquetas personalizables, facilitando la organización y búsqueda por categorías (ventas, soporte, reclamaciones, etc.).

Automatizaciones y chatbots: integrar un sistema de respuestas automáticas para gestionar las primeras interacciones con los clientes fuera del horario de atención o mientras no haya agentes disponibles.

Integración con múltiples canales: ampliar el sistema para soportar otros canales de comunicación además de WhatsApp, como Telegram, Facebook Messenger o Instagram Direct, convirtiendo PowerChat en una plataforma omnicanal.

Mejoras en el dashboard: añadir métricas de tiempo de respuesta, satisfacción del cliente, y exportación de informes en formato PDF o Excel.

Tests automatizados: implementar una suite completa de tests unitarios y de integración tanto en el backend (con Jest o Vitest) como en el frontend (con Vue Test Utils y Cypress para tests end-to-end).

CONCLUSIONES

El desarrollo de PowerChat ha permitido aplicar de forma integral las competencias adquiridas durante el Grado Superior en Desarrollo de Aplicaciones Web, abordando un proyecto completo que abarca desde el análisis de requisitos hasta el despliegue de una aplicación web funcional.

Desde el punto de vista técnico, el proyecto ha demostrado la viabilidad de construir una plataforma de gestión multiagente de WhatsApp utilizando tecnologías modernas del ecosistema JavaScript. La combinación de Vue 3 con Composition API en el frontend y Node.js con Express en el backend ha permitido desarrollar una aplicación coherente con un único lenguaje de programación en todo el stack. La integración de WebSockets mediante Socket.IO ha sido clave para conseguir la experiencia de tiempo real que requiere una herramienta de mensajería profesional.

Uno de los aprendizajes más significativos ha sido la implementación del sistema de autenticación y autorización. Si bien ya tenía experiencia previa con JWT gracias a mi trabajo profesional, este proyecto me ha permitido profundizar en otros aspectos de seguridad como el cifrado de contraseñas con bcrypt, el control de acceso basado en roles y la protección de webhooks mediante verificación de cabeceras secretas.

La experiencia profesional previa en una empresa del sector ha sido fundamental para el éxito del proyecto. No solo ha facilitado la fase de análisis al conocer de primera mano las necesidades reales de los usuarios, sino que también ha guiado las decisiones de diseño y arquitectura hacia soluciones probadas en entornos de producción.

El resultado es una aplicación funcional, segura y responsiva que cumple con todos los requisitos planteados en el anteproyecto. Si bien existen múltiples líneas de mejora identificadas en la sección de trabajos futuros, el alcance actual del proyecto demuestra las capacidades necesarias para abordar desarrollos web complejos con estándares profesionales.

REFERENCIAS

Cada referencia se acompaña de un texto descriptivo indicando el apartado del proyecto en el que se ha utilizado.

Aplicado en el desarrollo del backend con Node.js y Express.

OpenJS Foundation. (s. f.). Node.js Documentation. Recuperado de <https://nodejs.org/docs/latest/api/>

Aplicado en el desarrollo del backend con Node.js y Express.

StrongLoop/Express. (s. f.). Express.js Documentation. Recuperado de <https://expressjs.com/>

Aplicado en la gestión de la base de datos y el esquema de datos.

Prisma. (s. f.). Prisma Documentation. Recuperado de <https://www.prisma.io/docs>

Aplicado en la comunicación en tiempo real mediante WebSockets.

Socket.IO. (s. f.). Socket.IO Documentation. Recuperado de <https://socket.io/docs/v4/>

Aplicado en el desarrollo del frontend como framework principal.

Vue.js. (s. f.). Vue.js 3 Documentation. Recuperado de <https://vuejs.org/guide/introduction.html>

Aplicado en la herramienta de construcción del frontend.

Vite. (s. f.). Vite Documentation. Recuperado de <https://vitejs.dev/guide/>

Aplicado en la gestión del estado global del frontend.

Pinia. (s. f.). Pinia Documentation. Recuperado de <https://pinia.vuejs.org/>

Aplicado en el diseño responsivo y la maquetación de la interfaz.

Tailwind Labs. (s. f.). Tailwind CSS Documentation. Recuperado de <https://tailwindcss.com/docs>

Aplicado en las visualizaciones gráficas del dashboard.

Chart.js. (s. f.). Chart.js Documentation. Recuperado de <https://www.chartjs.org/docs/latest/>

Aplicado en la autenticación del sistema.

Auth0. (s. f.). JSON Web Tokens Introduction. Recuperado de <https://jwt.io/introduction>

Aplicado en la gestión de la base de datos.

Oracle Corporation. (s. f.). MySQL 8.0 Reference Manual. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>

Aplicado como referencia del estado del arte y herramienta similar.

Whanmo. (s. f.). Whanmo – WhatsApp multiagente para empresas. Recuperado de <https://whanmo.com/>